Mochi: A Baseline-Aware PC Health Buddy

Operating Systems case study · Oct 16, 2026 · @Royce

1. Project title

Mochi: A Baseline-Aware PC Health Buddy. A desktop companion that learns what normal looks like on one PC, then explains in plain words when something is off, why, and what to do.

Team member	Registration number
Royce B Selvaraj	24BCB0078
Srujan Buddharaju	24BDS0347

2. Introduction

Mochi is a Windows 11 tray app that compares every reading with this PC's own normal for that hour and activity. It lives in the system tray as a rice-cake mascot whose mood shows the machine's health: puffy when memory is full, sweaty when the CPU runs hot, dizzy when the system is swapping, grumpy when a drive is nearly full.

Under the mascot sit three serious jobs, each built on operating-system concepts:

- **Learn a personal baseline** of CPU, memory, paging, disk, temperature and battery drain for each hour of the week and each activity (browsing, coding, gaming, media, office, idle).
- **Detect and explain anomalies**: flag sustained unusual usage, name the responsible process, and diagnose the OS-level cause (memory pressure, a runaway process, disk I/O wait, thermal throttling, a memory leak, a filling disk).
- **Notice when the PC gets slower** over time, using app startup times and small repeatable benchmarks, and suggest a guided check-up.

3. Problem statement

Existing monitors show raw numbers but never say whether those numbers are normal for this PC, or what to do about them.

- **Fixed thresholds don't fit real people.** "Warn at 80% CPU" spams a gamer or a student compiling code, yet stays silent when an idle laptop sits at 40% because of a stuck background process.
- **The real cause is often invisible.** A PC can feel sluggish at 30% CPU because the real problem is memory pressure (paging), disk I/O wait or thermal throttling.
- **Slow decline goes unnoticed.** Startup bloat, a filling disk or a slow memory leak make the PC a little slower each week.
- **Warnings without advice are useless.** "High memory usage" doesn't say which app is responsible or what would help.

Users need a monitor that compares against their own normal behaviour, explains the OS-level cause in plain language, and gives one clear, safe action.

4. Objectives

1. Collect system-wide and per-process metrics (CPU, memory, paging, disk busy/latency, throttling, temperature, battery drain) every 5 seconds, using under 2% CPU and 100 MB RAM for Mochi itself.
2. Learn contextual baselines (48 hour-of-week slots × 7 activities) with an exponentially weighted moving average (EWMA), and flag only sustained anomalies using z-scores instead of fixed thresholds.
3. Diagnose the likely OS-level cause of each anomaly and attribute it to the responsible app, grouping multi-process apps such as Chrome.
4. Detect slow decline through app startup timing and micro-benchmarks, combined into a Health Score, and offer a guided PC check.
5. Present all of this through a calm, accessible interface: a mascot, gentle notifications with one tip each, and safe one-click actions.

5. Survey on existing work

Every common tool shows what is happening now; none of the desktop tools tells a non-expert whether it is normal for their PC or what to do.

Tool	What it does well	What it lacks for this problem
Windows Task Manager	Per-process CPU, memory, disk; startup impact	No history, no notion of "normal", no cause or advice
Resource Monitor / Performance Monitor	Detailed counters (paging, disk queue, latency)	Expert-only; user must know which counter matters

Tool	What it does well	What it lacks for this problem
Sysinternals Process Explorer	Process tree, handles, signatures	Snapshot view; no trends or alerts
Reliability Monitor	Daily history of crashes and installs	No performance metrics or anomaly detection
htop / btop (Linux)	Fast live view of processes and load	No baseline, alerts or diagnosis
Glances, Netdata	Many metrics, dashboards; Netdata adds anomaly detection	Built for servers and admins; no plain-language advice
Prometheus + Grafana	Time series, alert rules	Needs setup; rules use fixed thresholds written by experts

The statistical idea Mochi borrows is the EWMA control chart from statistical process control: a smoothed mean and variance that adapt slowly, with alerts when readings stay several standard deviations away. This survey is based on our own use of these tools; no external sources were consulted.

6. Existing and proposed systems

The proposed system replaces fixed thresholds and raw numbers with a learned, per-context normal and a plain-language diagnosis.

Aspect	Existing systems	Proposed system (Mochi)
What counts as a problem	Fixed thresholds (e.g. 80% CPU)	Sustained deviation from this PC's normal for this hour and activity, plus practical floors and absolute limits
Context	None	48 hour-of-week slots × 7 activities, with fallback to broader buckets
Single spikes	Often alert	Ignored; alert only when ≥ 80% of samples stay high for 2 or 5 minutes
Cause	Not given	Ordered rules: memory pressure, runaway process, disk I/O, leak, memory hog, throttling, storage, idle CPU, battery

Aspect	Existing systems	Proposed system (Mochi)
Multi-process apps	30 separate chrome.exe rows	One app, memory summed without double-counting shared pages
Advice	None	One tip per alert, safe actions (close gracefully, lower priority, open location)
Learning from the user	None	"I meant to do that" removes the episode from the baseline and quiets similar alerts for 30 days
Slow decline	Not tracked	App startup timing, unbuffered benchmarks, Health Score out of 100
Privacy	Varies	Fully local, no network code, no window titles or command lines stored

7. Methodology with flowchart

Every 5 seconds a sample flows from the OS through detection to Mochi; slower jobs (trends, benchmarks) run on their own schedule.

```
flowchart TD
    A[Every 5 s: read system counters and all processes] --> B[Group
processes into apps; tag hour slot and activity]
    B --> C{Baseline for this context trusted?}
    C -- no --> D[Check absolute limits only: disk full, temperature,
critical memory]
    C -- yes --> E[z-score vs EWMA mean and sigma, plus practical floors]
    E --> F{High for at least 80% of 2 or 5 minutes?}
    F -- no --> G[Normal reading: queue a delayed baseline update]
    F -- yes --> H[Diagnose cause with ordered rules; name the culprit app]
    D --> H
    H --> I[Alert episode: open, escalate, cooldown, snooze or Shh]
    I --> J[Mood, popup with one tip, safe action]
    J --> K{User says I meant to do that?}
    K -- yes --> L[Drop episode from baseline; keep quiet for 30 days]
    K -- no --> M[Store in SQLite; show on Timeline]
```

1. **Collect**: one NtQuerySystemInformation call for all processes plus performance counters for paging, commit, disk and throttling.
2. **Contextualise**: 48 hour-of-week slots × 7 activities from the foreground app and idle time.

3. **Learn**: EWMA mean and variance per context, updated only from normal readings, committed after 20 minutes.
4. **Detect**: sustained z-scores (Psst z > 2 for 2 min, Heads up z > 3 for 5 min, Uh-oh z > 4 for 5 min), absolute limits, and trend detectors for leaks and filling disks.
5. **Diagnose and alert**: first matching rule decides the cause and tip; one episode per culprit.
6. **Validate**: unit tests, simulated scenarios, light real fault injection, and UI tests.

8. Software and hardware requirements

Mochi runs on any Windows 10/11 PC without admin rights; temperature and SMART disk details are optional extras that need admin.

Software	Version used	Purpose
Windows	11 Pro (build 26200)	Target OS; NT system calls and performance counters
Python	3.12	Language
psutil	7.2	CPU, memory, disks, battery, process actions
pywin32	312	Performance counters (PDH), file version info, WMI
ctypes (built in)	3.12	NtQuerySystemInformation, CallNtPowerInformation, idle time, foreground window
PySide6 (Qt)	6.12	Tray app and dashboard UI
pyqtgraph + NumPy	0.14 / 2.5	Live graphs with the baseline band
SQLite	built in	Samples, baselines, alerts (WAL mode)
pytest, pytest-qt	9.1 / 4.5	Unit, end-to-end and UI tests
Git	2.53	Version control

Hardware	Minimum	Development and test machine
CPU	Any x64, 2 cores	Intel Core i9-14900HX, 24 cores / 32 threads
Memory	4 GB	32 GB

Hardware	Minimum	Development and test machine
Storage	1 GB free (data stays under 500 MB)	1 TB NVMe SSD
Other	None	Laptop battery (for the battery rule), NVIDIA RTX 4070 (not monitored)

9. System architecture

A collector thread produces samples into a bounded queue and a processing thread consumes them (producer-consumer), so collecting never waits on analysis or the UI.

```
flowchart TD
    OS[Windows: NtQuerySystemInformation, PDH counters,
CallNtPowerInformation, idle and foreground window] --> COL[Collector thread:
samples every 5 s]
    COL -- bounded queue --> PROC[Processing thread]
    PROC --> BASE[Baselines: EWMA per hour slot and activity]
    PROC --> DET[Detector: sustained z-scores, absolute limits, leak and disk
trends, runaway rule]
    DET --> DIAG[Diagnosis engine: ordered rules, culprit app]
    DIAG --> ALR[Alert manager: episodes, cooldown, snooze, Shh, feedback]
    PROC --> DB[(SQLite store: samples, baselines, alerts, benchmarks)]
    TASK[Startup watcher and benchmark subprocess] --> DB
    DB --> HS[Health Score and check-up]
    ALR --> UI[PySide6 UI: tray mood, popups, Home, Timeline, Detective,
Check-up, Diary, Settings]
    HS --> UI
    UI -- I meant to do that --> BASE
    SUP[Supervisor thread] -. restarts dead workers .-> COL
    SUP -.-> PROC
```

An SQLite writer thread batches commits every 30 s in WAL mode, so a hard kill never corrupts the database.

10. Implementation timeline

Six weeks, with the core demo complete at the end of week 4 so there is always a working build.

```
gantt
    dateFormat  X
    axisFormat  W%s
    section Core
    Collector and storage (start daily data collection) :1, 2
    Baselines and detection, unit tests :2, 3
    Diagnosis, alerts, notifications :3, 4
    UI, mascot moods, Home, Timeline, Detective :4, 5
    section Stretch
    Startup timing, benchmarks, Health Score, check-up :5, 6
    Quirks, fault-injection tests, report, demo rehearsal :6, 7
```

Week	Work	Gate
1	Collector and SQLite storage; start collecting on our own PC	Real data collected daily
2	Contextual buckets, EWMA, z-scores, sustained rule, unit tests	
3	Diagnosis rules, process attribution, notifications	
4	UI and mascot: Home, Timeline, Detective	Core demo works end to end
5	Startup timer, benchmarks, Health Score, guided check	
6	Diary and achievements, fault-injection tests, report, demo rehearsal	Ready to present

11. Expected outcomes

The working build already meets its detection and overhead targets in tests; the false-alarm rate on a real week of use is still to be measured.

Outcome	Target	Result so far
Memory hog named with the right cause	Popup within 7 min	Passed (simulated); Puffy mood within 30 s
Runaway process, one core or all cores	Within 6 min	Passed (simulated); caught on this PC with a real CPU burner
Memory leak, 50 MB/min	Within 20 min, rate ± 20%	Passed (simulated); real leak script detected

Outcome	Target	Result so far
Disk bottleneck / disk filling	Within 5 min / 1 min	Passed (simulated); real disk script attributed
Normal heavy use in its usual slot	No alert	Passed (gaming at 9 pm)
False alarms	< 1 per day	Simulated week passed; real week pending
Mochi's own overhead	< 2% CPU, < 100 MB	Passed for the background engine; UI to be measured
Reading all processes	Cheap	One system call vs about 430 ms of CPU per scan with psutil
Test suite	All pass	451 tests: unit, simulated end-to-end, real-machine, UI

For users the outcome is fewer, better alerts: each one says what is happening, why, and one thing to do, in Mochi's voice with the number behind it.

12. Future scope

- **GPU and network monitoring**: GPU load and VRAM (Windows GPU Engine counters) and network throughput, both out of scope today.
- **Richer Linux support**: pressure stall information (/proc/pressure) and per-process I/O, plus a Wayland-friendly idle and foreground source.
- **A privileged helper service** for temperature and SMART data, so the app itself never needs admin.
- **Smarter baselines**: seasonal models and per-app baselines, learning faster from fewer days.
- **Per-app boot impact**, measured from the first minutes after sign-in.
- **Packaging**: a signed installer (Mochi.exe) with auto-update and uninstall.
- **Evaluation**: a real week-long false-alarm study and a small user study on how understandable the tips are.

13. Challenges and ethical considerations

Technical challenges we hit and how we solved them

Challenge	Solution
A plain EWMA absorbs a sustained anomaly within about a minute, so alerts never fire	Anomalous readings never enter the baseline; updates are committed after a 20-minute delay
Many key metrics (paging, I/O wait, throttling) are Linux-only in psutil	Windows performance counters (PDH) and NT system calls via ctypes
psutil's per-process scan cost about 430 ms of CPU	One NtQuerySystemInformation call reads every process
A stuck single-threaded script is only 3% of a 32-thread CPU	A per-process runaway rule: ≥ 90% of one core for 5 minutes
Chrome and VS Code run as dozens of processes	Group processes into apps; sum private bytes, not shared memory
Windows does maintenance work exactly when you're idle	Recognise Defender, Update and Search and explain them calmly
Temperature needs admin on Windows	Shown as "unavailable (needs admin)", never as 0
Python processes all look like python.exe	Name them by script, e.g. python.exe (memory_hog.py)

Ethical considerations

- **Privacy**: all data stays on the PC; the code imports no network library. Only app names are stored, never window titles, web addresses or command lines, which can hold passwords or tokens. Users can export or delete everything.
- **No false accusations**: Mochi never calls a program malicious; it says "unfamiliar" and shows the publisher and location.
- **Safe actions**: it never offers to end Windows system processes, checks a process is still the same one before acting, and asks apps to close nicely first so work isn't lost.
- **Calm by design**: a learning period, practical floors and a daily cap avoid alarm fatigue; Uh-oh is reserved for real trouble.
- **Accessibility**: text meets WCAG AA contrast (checked by tests), moods have text labels, and reduce-motion and high-contrast settings are respected.
- **Least privilege**: the app runs without admin rights.

14. Conclusion

Mochi shows that a PC monitor can judge readings against the PC's own normal instead of fixed thresholds, and explain the OS-level cause in plain words. The working build applies core OS concepts (system calls, processes and scheduling priority, producer-consumer threads, paging and memory pressure, disk I/O and file systems, least privilege) to a problem every user has. Tests show it catches memory hogs, runaways, leaks, disk bottlenecks and full disks within minutes while staying quiet during normal heavy use; the remaining step is a real week-long false-alarm study.